

Índice general

Resumen	I
1. Introducción general	1
1.1. Extracción de información musical	1
1.2. Contexto de aplicación	1
1.3. Objetivos y alcance	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	2
1.3.3. Alcance	2
1.4. Estado del arte	3
1.4.1. Procesamiento digital de señales y extracción a bajo nivel . .	3
1.4.2. Modelos preentrenados y soluciones comerciales	3
1.4.3. Síntesis comparativa	4
2. Introducción específica	5
2.1. Fuentes de datos de canciones	5
2.2. Representaciones numéricas de la música	5
2.3. Información semántica del audio	7
2.4. Modelos de inteligencia artificial utilizados	8
2.5. Despliegue e integración	9
3. Diseño e implementación	11
3.1. Conjuntos de datos y extracción de atributos	11
3.1.1. Análisis comparativo de conjuntos de datos públicos	11
GTZAN	11
FMA estándar	12
Million Song Dataset (MSD)	13
AudioSet	13
3.1.2. Reducción de dimensionalidad mediante estadísticas	13
3.1.3. Definición del espacio de variables	16
3.2. Ejecución de tareas sobre el conjunto de datos	17
3.2.1. Distribución y tratamiento de las variables objetivo	17
Loudness	17
Key y Mode	18
Tempo	19
Acousticness, Danceability, Energy, Instrumentalness, Liveness	21
Speechiness	20
Speechiness	21
Liveness	22
Top-5 primary mood	23
Top-5 secondary mood	24
3.2.2. Suggested genres de entrada	25

3.3. Selección de variables de entrada	26
3.3. Modelado y optimización e integración	27
Modelado mediante perceptrón multicapa (MLP)	27
4. Ensayos y resultados	28
Modelado mediante Gradient Boosting (LightGBM)	28
4.1. Pruebas funcionales del hardware	28
Optimización de hiperparámetros y regularización	28
3.4. Arquitectura de despliegue e integración	29
5. Conclusiones	29
3.4.1. Diseño del servicio predictivo	29
5.1. Conclusiones generales	29
3.4.2. Flujo de integración	29
5.2. Próximos pasos	29
4. Ensayos y resultados	31
Bibliografía	31
4.1. Pruebas funcionales del hardware	31
5. Conclusiones	33
5.1. Conclusiones generales	33
5.2. Próximos pasos	33
Bibliografía	35

Índice de figuras

1.1. Interfaz de consulta en Tunebat	4
2.1. Representaciones visuales del audio	6
2.2. Probabilidades de detección semántica	8
3.1. Algoritmo de extracción de atributos	14
3.2. Proceso de reducción de dimensionalidad	15
3.3. Distribución de loudness	18
3.4. Distribución de key y mode	18
3.5. Distribución de tempo	19
3.6. Múltiples roles de características perceptibles estimado	20
3.7. Distribución de peculiarities perceptibles	21
3.8. Distribución de liveliness	22
3.9. Distribución de primary mood	23
3.10. Distribución de primary and secondary mood	24
3.11. Distribución de frequency of suggested genres	25
3.12. Diagrama de despliegue e integración	30

Índice de tablas

2.1. Atributos representativos	7
3.1. Desglose del espacio de variables	16
3.2. Distribución porcentual de speechiness antes y después del pre- procesamiento	18 22
3.3. Distribución porcentual de liveness antes y después del preproce- samiento	23
3.4. Resumen de parámetros usados en la selección de variables.	26
3.5. Resumen de hiperparámetros óptimos por objetivo.	29

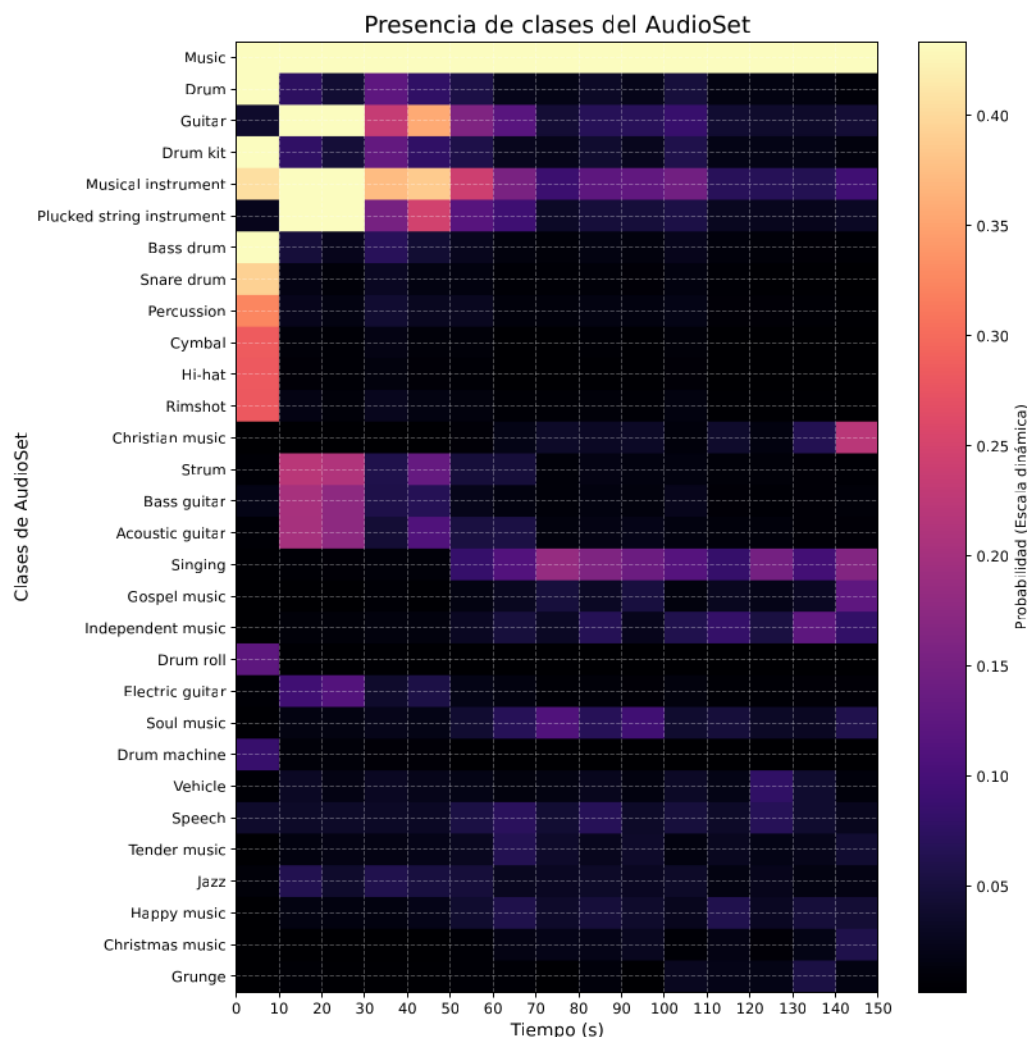


FIGURA 2.2. Mapa de calor de las características semánticas extraídas mediante la red CNN14.

2.4. Modelos de inteligencia artificial utilizados

Una vez obtenidos los descriptores numéricos y semánticos, es necesario aplicar algoritmos de predicción. Para este fin, se utilizaron dos enfoques principales:

- Algoritmos LightGBM: se integró la arquitectura LightGBM, basada en el ensamble de árboles de decisión. Esta herramienta destaca por su alta velocidad computacional y eficiencia en tareas generales de predicción [22]. En este caso también se usó Optuna para la búsqueda de parámetros óptimos [23].
- Redes MLP: los perceptrones multicapa son redes neuronales artificiales con capacidad para modelar relaciones complejas. Permiten, entre otras aplicaciones, predecir múltiples variables continuas de forma simultánea [24].

2.5. Despliegue e integración

La integración del sistema predictivo requirió del despliegue de una interfaz de programación de aplicaciones. Para su construcción se utilizó FastAPI [24] como

2.5. Despliegue e integración

marco de trabajo) para el desarrollo de servicios web de alto rendimiento basados en Python [25].

La integración del sistema predictivo requirió del despliegue de una interfaz de programación de aplicaciones. Para su construcción se utilizó FastAPI [25] como marco de trabajo. Esta herramienta facilita el desarrollo de servicios web de alto rendimiento basados en Python [26]. El servicio fue contenedorizado con Docker [27]. El alojamiento remoto del producto se apoyó en los servicios de infraestructura de OCI (por su sigla en inglés, *Oracle Cloud Infrastructure*) [28]. Se provisionó una instancia de cómputo virtual en la nube para desplegar la aplicación y ejecutar los modelos predictivos. El uso de esta tecnología garantiza la disponibilidad continua del sistema.

Capítulo 3

Diseño e implementación

Este capítulo detalla el diseño y la implementación del sistema predictivo. Se analizan los conjuntos de datos utilizados, la extracción de atributos del audio y el tratamiento de datos según las distribuciones de las variables objetivo. Asimismo, se describen las técnicas de reducción y selección de variables, el proceso de optimización de los modelos y los resultados de la iteración productiva. Finalmente, se presenta la arquitectura desarrollada para el despliegue y la integración del sistema en un entorno operativo.

3.1. Conjuntos de datos y extracción de atributos

3.1.1. Análisis comparativo de conjuntos de datos públicos

La definición de la estrategia para la extracción de atributos se fundamenta en una revisión de conjuntos de datos públicos estandarizados en el ámbito MIR. El objetivo de este análisis previo fue entender cómo repositorios de referencia, como FMA y Million Song Dataset [29], representan y estructuran los datos musicales, como método para establecer los criterios de diseño del conjunto de datos propio.

En un comienzo se contempló el uso de algunos de estos conjuntos de datos, pero se encontró que presentan varias limitaciones frente a la solución propuesta: algunos restringen el análisis a segmentos cortos de audio, mientras que otros requieren tamaños de almacenamiento excesivamente pesados o estructuras de datos difíciles de replicar durante el proceso de inferencia y dependientes de las versiones de las bibliotecas de extracción.

Por otro lado, el análisis resultó positivo; estas restricciones motivaron que el proceso de extracción se ejecute directamente sobre el catálogo privado para eliminar las limitaciones, y su análisis permitió conocer las variables numéricas en la música, cómo se segmentan las canciones, cómo se reduce la dimensionalidad y hacer los primeros experimentos predictivos de las variables objetivo requeridas.

GTZAN

Este conjunto de datos se considera el equivalente al MNIST [38] para el análisis de audio [39], dada su adopción histórica como estándar de evaluación en tareas de clasificación de géneros musicales. Con el análisis de este conjunto de datos se pudo identificar los siguientes puntos:

- Variables predictivas principales: tempo, representaciones chroma [18], amplitud media cuadrática (RMS) [17], descriptores espectrales [32] y coeficientes MFCC.
- Estructura: 1 000 archivos de audio en crudo limitados a 30 segundos, acompañados de metadatos tabulares e imágenes de espectrogramas de Mel en su versión distribuida mediante Kaggle [33]. Todas estas representaciones fueron extraídas con la biblioteca Librosa. Las tablas contienen los atributos calculados sobre la pista completa y también sobre divisiones de 3 segundos para incrementar el volumen de datos de entrenamiento. En ambos escenarios se aplica una reducción de dimensionalidad temporal: las variables se calculan inicialmente sobre micro-ventanas de milisegundos y luego se colapsan a valores estáticos mediante estadísticos de agregación, como la media y la varianza.
- Variables objetivo principales en Plusyc: se podría relacionar con *suggested genres*.
- Desventajas: provee segmentos cortos de audio, lo que limita la representatividad de la información musical a nivel de canción; y solo abarca 10 géneros musicales, mientras que Plusyc trabaja con más de 500.

FMA estándar

Se analiza la versión estándar de FMA, diseñada para superar las limitaciones de tamaño y acceso a las colecciones completa y mediana. Se encuentra:

- Variables predictivas principales: representaciones chroma, Transformada de Q Constante (CQT) [16], tonnetz, MFCC, RMS, descriptores espectrales y tasa de cruces por cero (ZCR).
- Estructura: ofrece recortes de audio de 30 segundos para sus subconjuntos estándar, acompañados de matrices de atributos precomputados distribuidos en formato tabular (CSV). Los atributos fueron extraídos con Librosa y posteriormente colapsados mediante estadísticos de agregación, como la media, desviación estándar, asimetría y curtosis, para obtener valores estáticos por pista.
- Metadatos contextuales: información tabular adicional que incluye álbum, artista, popularidad, fechas de lanzamiento y ubicación geográfica.
- Variables objetivo principales: géneros musicales estructurados en una taxonomía jerárquica de clases y para un subconjunto de aproximadamente 13 000 pistas, proporciona atributos como *danceability*, *energy*, *valence*, *speechiness* y *liveness*.
- Desventajas: el análisis está restringido a segmentos de 30 segundos en la versión estándar. Las versiones media y completa exceden las capacidades de procesamiento de la infraestructura actual de la empresa. Además, la incertidumbre sobre las versiones exactas de Librosa utilizadas originalmente dificulta la replicabilidad técnica sobre canciones nuevas. Finalmente, la información etiquetada de 13 000 pistas es mucho menor que la de más de 60 000 de Plusyc.

Million Song Dataset (MSD)

Desarrollado por la Universidad de Columbia y The Echo Nest, este conjunto escaló la investigación de información musical a nivel industrial. Se describen sus componentes principales:

- Variables predictivas principales: representaciones de tono (chroma y pitch) [18, 16] y descriptores de timbre [32], ambos calculados a nivel de segmento.
- Estructura: colección de metadatos y descriptores precomputados empaquetados en archivos HDF5.
- Método de extracción: a diferencia de la segmentación por ventanas fijas, el MSD utiliza una división adaptativa basada en eventos. Esto fragmenta la canción en segmentos de duración variable. Para cada uno de estos segmentos, se calculan coeficientes de tono y timbre, y se genera una matriz de atributos alineada con la estructura temporal de la pista.
- Variables objetivo principales: proporciona atributos musicales fundamentales como *tempo*, *loudness*, *key* y *mode*.
- Desventajas: los descriptores se generaron con algoritmos propietarios de The Echo Nest, lo que complica procesar la segmentación por eventos en canciones nuevas de forma idéntica para realizar el proceso de inferencia.

AudioSet

Este conjunto masivo establece un estándar para la clasificación de eventos sonoros generales:

- Estructura: conjunto de identificadores de videos de YouTube y marcas de tiempo que referencian segmentos de 10 segundos, anotados bajo una ontología de 527 clases.
- Variables predictivas principales: el corpus original no extrae descriptores tradicionales. Su distribución oficial provee embeddings precomputados mediante el modelo VGGish [34], que sirven como entrada rápida para entrenar clasificadores de terceros sin necesidad de descargar el audio original.
- Variables objetivo principales: etiquetas multiclase que identifican la presencia de instrumentos musicales, como tipos de voces (habla, canto) y ambientes acústicos.
- Desventajas: el enfoque en sonidos generales limita su precisión para los atributos musicales.

Cabe mencionar que, a pesar de que en algunas de las fuentes de datos públicas se incluye información del contexto como la región del artista, el año de lanzamiento, el álbum o las letras, en esta solución se trabaja exclusivamente con información musical que se extrae del audio.

3.1.2. Reducción de dimensionalidad mediante estadísticas

En el ámbito del procesamiento de señales, la información de un archivo de audio se representa como una serie de tiempo multivariada: una secuencia de puntos

Algunos experimentos preliminares sirvieron para validar que esta representación permitía capturar la información esencial de la canción de manera eficiente y compacta. El algoritmo de extracción, es el resultado de pruebas con distintas configuraciones de variables, estadísticos y métodos de segmentación.

3.1.3. Definición del espacio de variables

El procesamiento computacional sobre el catálogo de canciones requirió más de una semana de ejecución para procesar y extraer la información de más de 60 000 canciones completas. El alto costo computacional se justificó por la necesidad de garantizar la disponibilidad de información para los experimentos de modelos predictivos de múltiples características musicales.

La aplicación del algoritmo final generó un conjunto de datos masivo compuesto por 9590 variables, resumido en la tabla 3.1.

TABLA 3.1. Desglose estructural de las 9590 variables extraídas por canción.

Categoría de extracción	Atributos base	Cant.	Estadísticos aplicados	Cant.	Total
Representaciones DSP (Librosa)	MFCC (20), Deltas (40) [17], Centroide, Ancho de banda, Rolloff, Contraste (7) [32], Flatness y Flux [32], Chroma (12), Tonnetz (6), RMS, ZCR.	92	Media, Desv. Estándar, Asimetría, Curtosis, Percentiles (10, 50, 90).	7	644
Atributos globales y ritmo	LUFS [35], Entropía espectral [17], Tempo, Complejidad dinámica [17], Tasa y estadísticos de onset (4) [16], Pitch, Fracción de voz [36], Claridad tonal (2) [16], Factor de cresta [32], Desviación de latidos [16].	14	Cálculo único por pista completa.	1	14
Estimación tonal (Perfiles)	Vectores de correlación Krumhansl (Mayor/Menor) y Temperley (Mayor/Menor) [16].	48	Cálculo único por pista completa (4 perfiles $\times$ 12 semitonos).	1	48
Embeddings semánticos (PANNs)	Vector latente de la capa previa a la clasificación de la red CNN14.	2048	Media, Máximo, Desv. Estándar, Percentil 90.	4	8192
Probabilidades (PANNs)	Clases filtradas (Voz, Instrumentos, Géneros, Público en vivo, Distorsión).	173	Media, Máximo, Desv. Estándar, Percentil 90.	4	692

Total de variables por canción: 9590

3.2. Ejecución de tareas sobre el conjunto de datos

Esta sección presenta las variables objetivo y sus distribuciones en el contexto de Plusyc, el tratamiento aplicado a los datos y las técnicas de selección y reducción de variables predictivas.

Las variables objetivo se dividen en tres grupos: básicas, perceptibles y género-estados de ánimo sugeridos. Y se definen en la tabla 3.2.

3.2.1. Distribución y tratamiento de las variables objetivo

Durante la extracción de variables descrita en el algoritmo de la sección 3.1.2, se aplicaron dos estrategias adicionales para el tratamiento de los datos previos al modelado:

Durante la extracción de variables descrita en el algoritmo de la sección 3.1.2, se aplicaron dos estrategias adicionales para el tratamiento de los datos previos al modelado:

- **Recorte de silencios y manejo de valores nulos:** tras la primera iteración de extracción de variables (DSP), se detectó la presencia de valores nulos y ceros en algunas variables. El análisis exploratorio reveló que eran generados por las colas de silencio o fade-outs prolongados al final de las pistas. Para corregirlo, se aplicó un algoritmo de recorte de segmentos con amplitud inferior a -60 dB. Esta operación eliminó la gran mayoría de los nulos. El restante se concentró exclusivamente en un 6 % del descriptor `spectral_entropy_global`. Matemáticamente, el cálculo de la entropía espectral requiere normalizar la magnitud del espectro para interpretarlo como una distribución de probabilidad. En ventanas de análisis correspondientes a silencios absolutos, la energía espectral es nula [17]. Matemáticamente, el cálculo de la entropía espectral requiere normalizar la
- **Filtrado del etiquetado manual:** al analizar las variables objetivo, se identificó un cambio en una fecha específica en la fuente de los datos: hasta mediados de diciembre de 2024, los valores provenían de la integración con una API externa. Los registros generados durante 2025, ingresados manualmente por el equipo de curadores, introdujeron error, caracterizado por la presencia de valores por fuera del rango en variables discretas, valores extremos en cero y uno en variables continuas. Para proteger el entrenamiento de los modelos, el conjunto de datos de análisis fue truncado para conservar únicamente los registros fiables hasta diciembre de 2024. Resulta en el uso de un 92,5 % aproximado de las canciones del catálogo de Plusyc y aplica
- **Filtrado del etiquetado manual:** al analizar las variables objetivo, se identificó un cambio en una fecha específica en la fuente de los datos: hasta mediados de diciembre de 2024, los valores provenían de la integración con una API externa. Los registros generados durante 2025, ingresados manualmente por el equipo de curadores, introdujeron error, caracterizado por la presencia de valores por fuera del rango en variables discretas, valores extremos en cero y uno en variables continuas. Para proteger el entrenamiento de los modelos, el conjunto de datos de análisis fue truncado para conservar únicamente los registros fiables hasta diciembre de 2024. Resulta en el uso de un 92,5 % aproximado de las canciones del catálogo de Plusyc y aplica

A continuación, se presentan las distribuciones de las variables objetivo después del filtrado y el preprocesamiento adicional aplicado: estados de ánimo donde la información no dependía de la API.

Loudness

A continuación se presentan las distribuciones de las variables objetivo después del filtrado y el preprocesamiento adicional aplicado.

El *loudness* define el volumen global promedio de la pista acústica en decibeles (dB) (ej. -5,2 dB o -12,4 dB). Los valores más negativos indican menor intensidad [37]. La figura 3.3 muestra la distribución de frecuencia de los valores del *loudness*. Se observa que presenta un sesgo negativo, con una concentración principal cerca de los -7 dB. Esto se explica dada la guerra del *loudness* [38], donde muchos artistas y productores buscan que sus canciones suenen con volumen alto, y plataformas como Spotify normalizan el *loudness* para evitar cambios abruptos en el volumen entre las canciones.

Desafíos:

TABLA 3.2. Definición, ejemplos y tipo de problema de aprendizaje supervisado por característica.

Característica	Definición	Ejemplos	Tipo de problema
Básicas			
Loudness	Volumen global promedio en decibeles, más negativa, más silenciosa (dB)	-5.2 dB, -12.4 dB	Regresión
Key	Tonalidad, estimado de la pista mapeada a números enteros (Pitch Class)	0 (Do), 1 (Do#), 2 (Re)	Clasificación
Mode	Modalidad (mayor o menor) del contenido melódico	1 (Mayor), 0 (Menor)	Clasificación
Tempo	Tempo global estimado en pulsaciones por minuto (BPM)	120, 85	Regresión
Perceptibles			
Acousticness	Confianza de que la pista es acústica (0,0 a 1,0).	0,95, 0,10	Regresión
Danceability	Confianza de que la pista es apta para bailar (0,0 a 1,0)	0,85, 0,20	Regresión
Energy	Medida de intensidad y actividad (0,0 a 1,0).	0,90	Regresión
Key y Mode			
Instrumentalness	Probabilidad de ausencia de contenido vocal (0,0 a 1,0)	0,90 (sin voz), 0,05 (cantada)	Regresión
Liveness	Confianza de grabación en vivo (0,0 a 1,0)	0,85 (concierto), 0,10 (estudio)	Regresión
Genre	Género musical	Chill EDM, House	Clasificación
Valence	Positividad musical transmitida (0,0 a 1,0).	0,90, 0,15	Regresión
Primary Mood	Estado de ánimo o emoción predominante.	Energizing, Lively, Empowering	Clasificación
Secondary Mood	Estados de ánimo complementarios.	Euphoric Energy, Energetic, Anxious	Clasificación
Suggested Genres	Género musical	Chill EDM, House	Clasificación

FIGURA 3.3. Distribución de loudness en el catálogo de Plusyc.

Se observan dos desafíos principales inherentes a la distribución:

- Existencia de asimetría negativa (sesgo a la izquierda).
- Presencia de valores extremos negativos (hasta -60 dB).

Esto potencialmente podría generar un error matemático que desvíe la atención del algoritmo durante el entrenamiento [24]. Se aplicaron las siguientes transformaciones como parte del preprocesamiento:

- Se aplicó una transformación logarítmica desplazada (Shifted Log-Transformation). Primero, se realizó una traslación de la escala ($x + 60, 1$) para garantizar que todos los registros fueran estrictamente mayores a cero.
- Posteriormente, se aplicó la función matemática $\log(1 + x)$ ('np.log1p'). Este preprocesamiento comprime la cola izquierda de la distribución, mitiga el sesgo original y estabiliza la varianza para mejorar el rendimiento del modelo de regresión [24].

Las variables *key* y *mode* representan la tonalidad predominante de la pista mapeada a clases de tono enteras (ej. 0 para Do, 1 para Do#) y su modalidad armónica (1 para Mayor, 0 para Menor) [39]. La figura 3.4 muestra la distribución conjunta de las frecuencias de los valores discretos de las 24 tonalidades de la música occidental. Se observa preferencia por tonalidades como Do mayor, Re mayor y Sol mayor, y menor frecuencia de canciones en Re sostenido mayor y menor.



FIGURA 3.4. Distribución conjunta de *key* y *mode* en el catálogo de Plusyc.

Algunos desafíos inherentes a la distribución:

- Existe un desbalance marcado en las tonalidades, lo que propicia que los modelos de clasificación colapsen hacia las clases mayoritarias [24].
- Los algoritmos de aprendizaje automático requieren que las variables categóricas (como las notas musicales) estén representadas en un formato numérico procesable.

Se siguieron las siguientes estrategias como parte del preprocesamiento:

- Para adaptar las etiquetas alfanuméricas de las tonalidades al entrenamiento, se aplicó codificación de etiquetas (Label Encoding).
- Para mitigar el desbalance sin eliminar información musical intrínseca, el problema se abordó directamente en la fase de modelado mediante la ponderación de clases (`class_weights='balanced'`).
- Existencia de asimetría negativa (sesgo a la izquierda) y presencia de valores extremos negativos (hasta -60 dB). Esto potencialmente podría generar un error matemático que desvíe la atención del algoritmo durante el entrenamiento [25].
- Se aplicó una transformación logarítmica desplazada (*Shifted Log-Transformation*).

Primero, se realizó una traslación de la escala ($y + 60,1$) para garantizar que todos los registros fueran estrictamente mayores a cero.

El *tempo* indica la velocidad global estimada de la pista en pulsaciones por minuto (ej. 120 BPM u 85 BPM) [37]. La figura 3.5 muestra la distribución de frecuencia de los valores del *tempo*. Se observa una concentración principal en el rango de los 90 a 130 BPM, con un pico destacado alrededor de los 120 BPM, predominante en la música popular contemporánea [16].

Key y Mode

La figura 3.4 muestra la distribución conjunta de las frecuencias de los valores discretos de las 24 tonalidades de la música occidental. Se observa preferencia por tonalidades como Do mayor, Re mayor y Sol mayor; y menor frecuencia de canciones en Re sostenido mayor y menor.

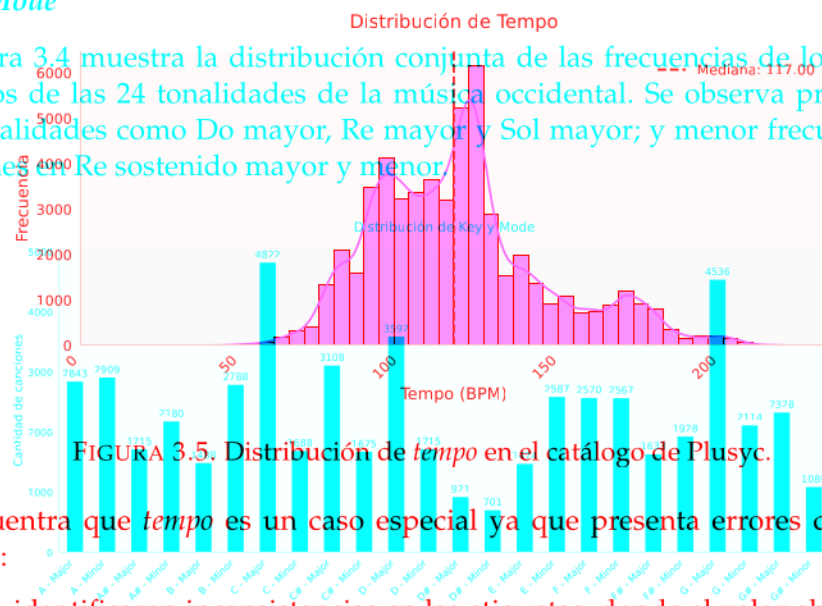


FIGURA 3.5. Distribución de *tempo* en el catálogo de Plusyc.

Se encuentra que *tempo* es un caso especial ya que presenta errores de octava métrica:

- Se identificaron inconsistencias en las etiquetas, donde el valor objetivo registrado representaba exactamente la mitad, el doble o proporciones fraccionarias (ej. 1.5x) del tempo real. Este es un problema documentado en la extracción de información musical conocido como error de octava (octave error) [16].

Desafíos:

- Este problema puede explicar la presencia de valores atípicos en los extremos de la distribución.
- La presencia de errores de octava complica el uso de algoritmos de regresión lineal ya que se observan múltiples relaciones lineales entre variables rítmicas predictivas y el tiempo global (ej. predecir 60 BPM para una pista etiquetada en 120 BPM).

Tratamiento:

Se contempla la evaluación de los algoritmos DSP de estimación de *tempo* integrados en las bibliotecas de DSP, esta comparación se muestra en la sección de validación.

- Para mitigar el desbalance sin eliminar información musical intrínseca, el

Acousticness, Danceability, Energy, Instrumentalness y Valence

Este grupo comprende variables perceptibles continuas evaluadas en el rango [0, 1]: *acousticness*: confianza acústica, *danceability*: bailabilidad métrica, *energy*: intensidad energética, *instrumentalness*: ausencia vocal y *valence*: positividad musical [37]. La figura 3.6 muestra las distribuciones de frecuencia de los valores de estas características, ya que afecta la familia armónica. La predicción de un top-k de *key* y *mode* conjuntos, generalmente muestran relaciones armónicas de la canción, esto sentido en el contexto de la ambientación.

Tempo

La figura 3.5 muestra la distribución de frecuencia de los valores del *tempo*. Se observa una concentración principal en el rango de los 90 a 130 BPM, con un pico destacado alrededor de los 120 BPM, predominante en la música popular contemporánea [16].

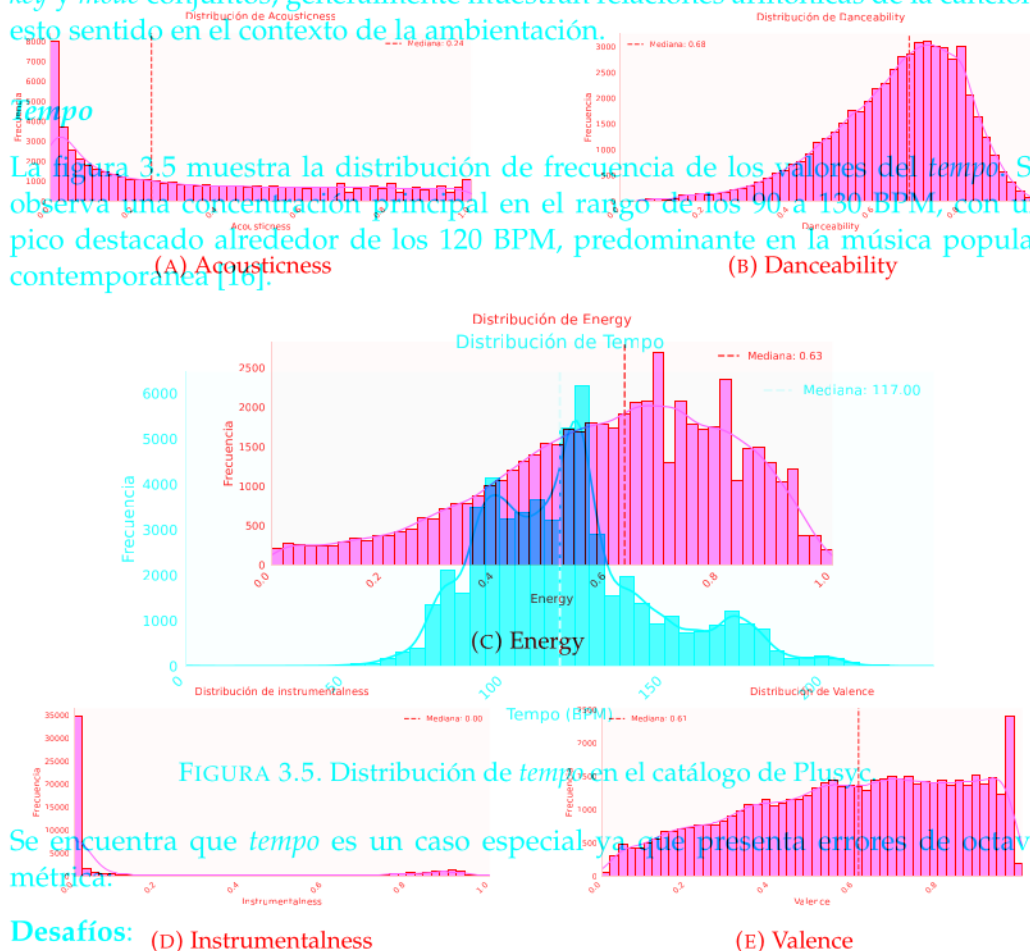


FIGURA 3.5. Distribución de *tempo* en el catálogo de Plusyc.

Se encuentra que *tempo* es un caso especial ya que presenta errores de octava métrica.

Desafíos:

- Se identificaron inconsistencias en las etiquetas, donde el valor objetivo registrado representaba exactamente la mitad, el doble o proporciones fraccionarias (ej. 1.5x) del tiempo real. Este es un problema documentado en la recuperación de información musical conocido como «error de octava» (*octave error*) [16].

Aunque todas comparten un rango de [0, 1], presentan comportamientos distintos: *danceability* muestra una distribución aproximadamente normal; *energy* presenta sesgo negativo; *valence* tiende a la uniformidad; mientras que *acousticness* y

de forma extrema, *instrumentalness*, exhiben un sesgo positivo con una concentración masiva de valores cercanos a cero.

El problema de múltiples características se aborda convencionalmente mediante redes neuronales, como el Perceptrón Multicapa (MLP), dada su capacidad para aprender de forma simultánea las relaciones complejas y dependencias no lineales entre ellas [24]. Bajo esta premisa, el preprocesamiento requiere prestar atención específica en:

- La heterogeneidad en las formas de las distribuciones y sus varianzas. En arquitecturas sensibles a la escala, como los MLP, las variables con mayor varianza empírica pueden dominar el cálculo de los gradientes, ralentizar o sesgar la convergencia de los pesos [24].
- Las MLP son sensibles a los valores nulos y están presentes en un 6 % de *spectral_entropy_global*.

Se implementaron las siguientes medidas:

- A diferencia del *loudness*, las variables objetivo ya se encuentran acotadas matemáticamente en el intervalo $[0, 1]$. Por tanto, no requirieron transformaciones, ya que este rango es compatible con las funciones de activación continuas, como la sigmoid, en la capa de salida de la red neuronal.
- Se garantizó la integridad de la matriz de características al imputar los nulos residuales con el valor cero y así reflejar la falta de información acústica.

Se contempla la evaluación de los algoritmos DSP de estimación de *tempo* integrado en la biblioteca de DSP de estimación de *tempo* de *Essentia*.

Finalmente, se aplica estandarización estadística (StandardScaler) a todo el conjunto de entrada para garantizar una media de cero y varianza unitaria ($\mu = 0, \sigma^2 = 1$). Este paso es necesario para homogeneizar el espacio de variables predictivas para el entrenamiento de la red neuronal [24].

La figura 3.7 muestra las distribuciones de frecuencia de los valores de estas características. Aunque todas comparten un rango de $[0, 1]$, presentan comportamientos distintos. El *speechiness* detecta la presencia de palabras habladas en la pista y se define en $[0, 1]$, donde valores altos indican contenido exclusivamente hablado y valores bajos indican música (ej. 0,85 para conversación, 0,05 para música y 0,5 es rap) [37]. La figura 3.7 muestra la distribución de frecuencia del *speechiness*.

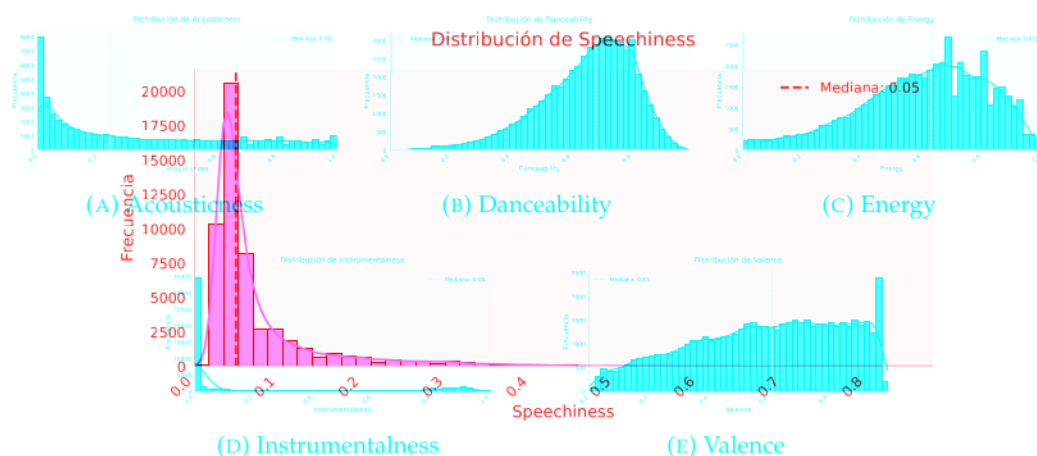


FIGURA 3.7. Distribución de *speechiness* en el catálogo de Plusyc.

FIGURA 3.7. Distribuciones de características perceptibles en el catálogo de Plusyc.

La distribución muestra la presencia de un sesgo positivo extremo, ya que el catálogo musical carece de pistas de palabra hablada pura y concentra los valores

muy cerca de cero. Los valores cerca del uno indican contenido exclusivamente hablado como podcast o streaming conversacional, ausentes en el catálogo de Plusyc. Para aprender de forma simultánea las complejas correlaciones y dependencias no lineales entre ellas [23]. Bajo esta premisa, el preprocesamiento requiere el mejor resultado experimental en términos de las métricas de evaluación se obtuvo al aplicar técnicas de submuestreo. Se discretizó el dominio de la variable en rangos estratégicos y se aplicó un submuestreo con tope por rangos para limitar la cantidad de registros en la moda, reducir el sesgo y conservar la totalidad de registros de la cola. Los rangos se presentan en la Tabla 3.2.

TABLA 3.2. Distribución porcentual de *speechiness* antes y después del preprocesamiento.

Intervalo	Descripción del Rango	Las MLP son sensibles a los valores nulos y el descriptor <i>spectral_entropy_global</i> (presente en un 6 % de la muestra).	
		% Original	% Final
[0, 00, 0, 05]	Menor presencia de canto	56,0 %	42,1 %
(0, 05, 0, 33]	Canto	41,9 %	42,1 %
(0, 33, 1, 00]	Rap	2,1 %	15,7 %
Total		100 %	100 %

Cabe mencionar que se aplicó muestreo estratificado a nivel de los rangos durante la división de conjuntos de entrenamiento y pruebas. Finalmente, se mitigó el ruido en las etiquetas al eliminar un porcentaje menor de registros contradictorios: se compararon los valores de la variable objetivo ya encuentran acotadas matemáticamente en el intervalo [0, 1]. Por tanto, no requirieron transformaciones, ya que este rango es compatible con las funciones de activación continuas (como la sigmoide) en la capa de salida una red neuronal. Finalmente, se aplica estandarización estadística (*StandardScaler*) a todo el conjunto de entrada para garantizar una media de cero y varianza unitaria ($\mu = 0, \sigma^2 = 1$). Este paso es necesario para homogeneizar el espacio de variables predictivas para el entrenamiento del modelo neuronal

Liveness

El *liveness* representa la confianza de que la pista fue grabada con público en vivo [0, 1] (ej. 0,85 para un concierto, 0,10 para un estudio) [37]. La figura 3.8 muestra la distribución de frecuencia del *liveness*. La figura 3.8 muestra las distribuciones de frecuencia del *speechiness*.

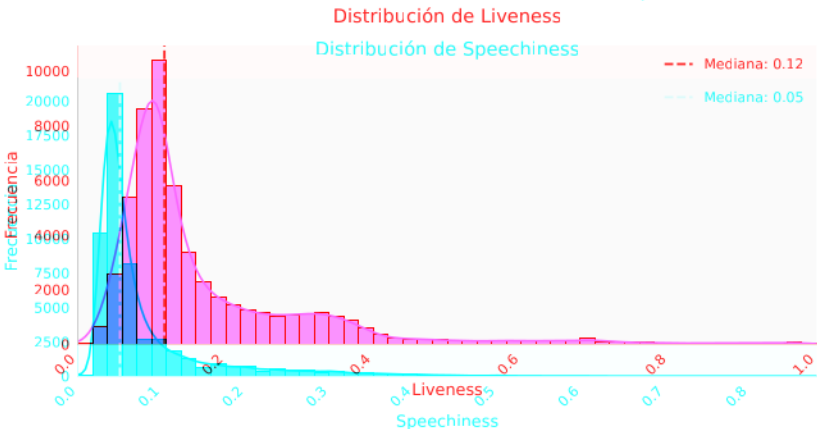


FIGURA 3.8. Distribución de *liveness* en el catálogo de Plusyc.
FIGURA 3.8. Distribución de *speechiness* en el catálogo de Plusyc.

Esta distribución, como en el caso anterior, también muestra la presencia de un sesgo positivo extremo, ya que concentra la inmensa mayoría de los registros por debajo de 0,2. Adicionalmente, se identificó un problema de ruido sistemático en las etiquetas manuales, donde pistas de estudio con una alta percepción de energía o intensidad eran catalogadas erróneamente como "vivo".

Se aplicó una técnica similar a la anterior para balancear el objetivo continuo, se dividió el dominio con base en la definición teórica de la característica y se aplicó un submuestreo con tope por rangos. Se aplicó reducción en la clase mayoritaria y en la zona intermedia para evitar sesgos y se conservan todos los registros minoritarios de la clase de interés: canción en vivo. El resultado de esta reestructuración se detalla en la Tabla 3.3.

TABLA 3.3. Distribución porcentual de liveness antes y después del preprocesamiento

Intervalo	Descripción del Rango	% Original	% Final
[0, 0, 0, 2)	Muy baja probabilidad de vivo	74,4 %	67,4 %
[0, 2, 0, 8)	Baja probabilidad de vivo	24,8 %	22,5 %
[0, 8, 1, 0]	Alta probabilidad de vivo	0,8 %	10,1 %
Total		100 %	100 %

FIGURA 3.9. Distribución de *liveness* en el catálogo de Plusyc.

Como en el caso de *speechiness*, también se aplica muestreo estratificado por rangos durante la separación de conjuntos de entrenamiento y pruebas. Y se implementó un filtrado cruzando la variable objetivo con la evidencia de presencia de audiencia extraída por la red PANNS, de las probabilidades de clases asociadas a multitudes, aplausos y aclamaciones. Se descartaron las pistas que presentaban discrepancias severas para eliminar ruido (ej. registros etiquetados como 'en vivo' sin evidencia de sonido de público, o viceversa).

La figura 3.10 muestra los *primary mood* de Plusyc y sus frecuencias.

El *primary mood* define el estado de ánimo o emoción predominante que transmite la pista (ej. *Energizing*, *Lively*, *Empowering*). La figura 3.9 muestra los *primary mood* de Plusyc y sus frecuencias.

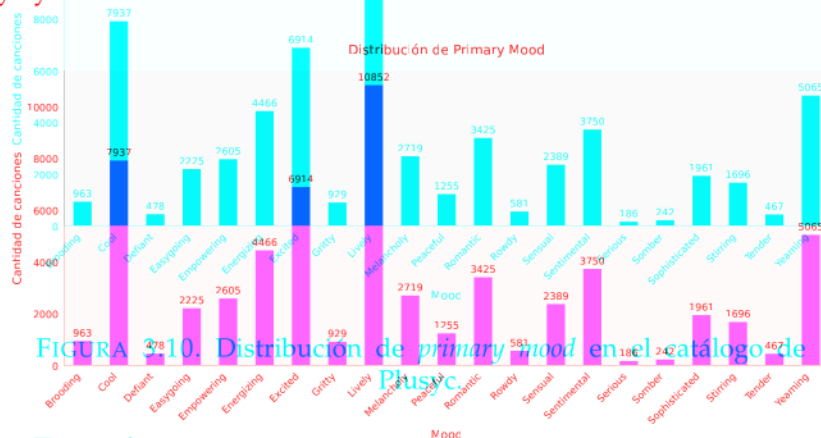


FIGURA 3.9. Distribución de *primary mood* en el catálogo de Plusyc.

Se observan los siguientes desafíos en esta variable categórica:

La figura 3.11 muestra los *primary mood* de Plusyc y sus frecuencias.

- Alta cardinalidad y un marcado desbalance de clases, donde ciertas emociones predominan debido a la orientación comercial y de ambientación del catálogo. Algunas clases se han dejado de usar en el pasado.

La figura 3.12 muestra las distribuciones de frecuencia de los *suggested genres*.

- La predicción se fundamenta exclusivamente en variables numéricas extraídas del audio, ya que se carece de información de lenguaje, como las letras de las canciones.
- Ambigüedad; incluso para un humano, algunos de los estados de ánimo pueden ser confusos.

Se aplicaron las siguientes transformaciones como parte del preprocesamiento:

- Se definió con el negocio prescindir de las clases minoritarias *Urgent*, *Aggressive*, *Upbeat* y *Fiery* que no son usadas para ambientación.
- Submuestreo de clases mayoritarias para equilibrar la distribución con techo de 2 000 muestras por clase.
- Aplicación de Label Encoding para traducir las categorías de texto a un formato numérico procesable.
- Para mitigar el desbalance, se aplicó la técnica de ponderación de clases (`class_weights='balanced'`) directamente durante la fase de modelado.

FIGURA 3.11. Distribución de *secondary mood* en el catálogo de Plusyc.

Top-5 secondary mood

El *secondary mood* identifica los estados de ánimo complementarios (ej. *Euphoric Energy*, *Energetic Anxious*). La figura 3.10 muestra las frecuencias de los *secondary mood* de Plusyc. Las clases más populares del catálogo son *Cheerful / Playful* con 4 322 muestras, *Happy Excitement* con 3 738 y *Sweet / Sincere* con 3 154.

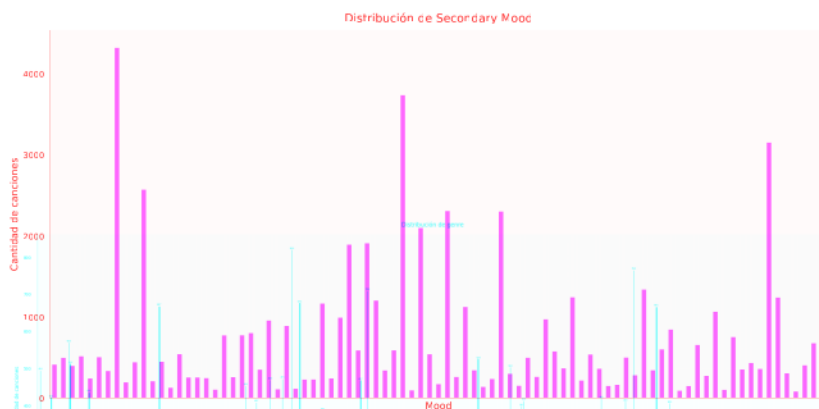


FIGURA 3.10. Distribución de frecuencias de *secondary mood* en el catálogo de Plusyc.

En este caso las dificultades son similares al caso de *primary mood*, en el desbalance de clases, pero con mayor cardinalidad, con 100 clases en total. Además de la delgada frontera en muchos de los estados de ánimo que incluso para el humano resultan confusos.

FIGURA 3.12. Distribución de *suggested genres* en el catálogo de Plusyc.

Medidas aplicadas en el preprocesamiento:

- Se define con el negocio prescindir de las clases minoritarias *Aggressive Power*, *Alienated / Brooding*, *Bittersweet Pop*, *Chaotic / Intense*, *Creepy / Ominous*, *Energetic Dreamy*, *Evocative / Intriguing*, *Focused Sparkling*, *Hard Dark Excitement*, *Heavy Triumphant*, *Hypnotic Rhythm*, *Mysterious / Dreamy*, *Wild / Rowdy* que no son usadas para ambientación en Plusyc.

■ Submuestreo de clases mayoritarias para equilibrar la distribución con techo de 1 000 muestras por clase.

3.2.2. Selección de variables de entrada

- Al igual que con el estado de ánimo principal, se aplicó Label Encoding para la adaptación numérica.

3.3. Modelado y optimización

- Se mantuvo la estrategia de balanceo (`class_weights='balanced'`) para evitar sesgos.

3.4. Arquitectura de despliegue e integración

Suggested genres

Los *suggested genres* indican las etiquetas de género musical asociadas a la pista (ej. *Chill House*, *EDM*). La figura 3.11 muestra las frecuencias de los *suggested genres*. Las clases más populares del catálogo son *EDM* con 819 muestras, *Pop Vocal* con 764 y *General Latin Pop* con 708.

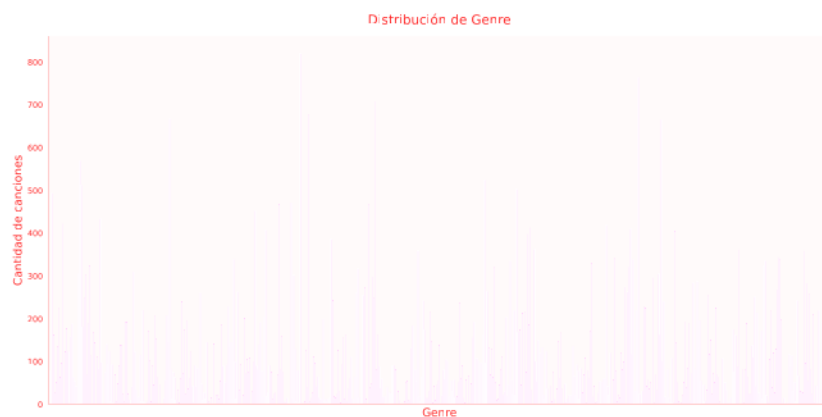


FIGURA 3.11. Distribución de frecuencias *suggested genres* en el catálogo de Plusyc.

Se observa la extrema diversidad de clases, 585 en total y muchas clases con muy pocas muestras, lo que genera una distribución de cola larga altamente desbalanceada. Por otro lado al revisar algunas clases aparecen géneros que no son géneros sino marcas o artistas como *Shakira*, *Juanes* o *Decathlon*. Otra dificultad radica en que al analizar solo el audio, se puede perder el contexto regional de géneros como *Rock*, *Latin Rock*, que podrían ser confusos en las predicciones.

Se aplicaron las siguientes estrategias:

- Se filtran géneros que no son géneros musicales, sino marcas o artistas.
- Se seleccionan un tope mínimo de 100 muestras por clase para el modelo. De las 585 clases totales, se seleccionan 200 clases que cumplen el criterio y se determinó que las demás quedan pendientes de alcanzar el mínimo de 100 muestras para ser incluidas en el entrenamiento.
- Submuestreo de clases mayoritarias para equilibrar la distribución con techo de 400 muestras por clase.
- Transformación de las categorías mediante Label Encoding.
- Delegación del manejo del desbalance al hiperparámetro de ponderación (`class_weights='balanced'`) durante el entrenamiento del clasificador.

- Estrategia de clasificación por top-k, para mitigar el problema de contexto al seleccionar los más probables.

3.2.2. Selección de variables de entrada

La aplicación del algoritmo de extracción generó un espacio predictivo masivo compuesto por 9590 variables. Modelar directamente sobre esta dimensionalidad introduce riesgo de sobreajuste y alto costo computacional [39, 40]. Por ello, la estrategia de diseño trasladó el esfuerzo hacia el proceso de selección de variables predictivas.

Para todas las variables objetivo, el proceso de selección se ejecutó estrictamente sobre los datos de entrenamiento (X_{train}) para evitar sobreajuste, mediante un flujo secuencial de tres etapas:

1. Filtro por varianza: se implementó `VarianceThreshold` [41] para evaluar cada característica de forma aislada y descartar aquellas constantes o sin varianza (umbral 0,00). Se aplicó un umbral de 0,01 en objetivos de alta cardinalidad para purgar características dispersas que carecen de valor predictivo [42].
2. Selección basada en árboles, con las siguientes características:
 - El cálculo de la importancia se realizó predominantemente mediante un entrenamiento simple, para priorizar la eficiencia computacional. Excepcionalmente, en las variables perceptibles continuas, se implementó validación cruzada (K-Fold con $K = 5$) para mitigar el sobreajuste al ruido en este tipo de etiquetas subjetivas [43].
 - La métrica utilizada fue la ganancia (Gain) para medir el aporte de las variables en las particiones de los árboles [44, 22].
 - Se conservaron únicamente las características con mayor importancia, responsables de explicar el 95 % de la ganancia acumulada del modelo.
3. Filtro por dependencia lineal: se calculó la matriz de correlación de Pearson para analizar las características en conjunto y eliminar aquellas que presentaban redundancia lineal [45, 42].

En la tabla 3.4 se resume la configuración del proceso y el total de variables predictivas resultantes para cada variable objetivo.

TABLA 3.4. Resumen de parámetros usados en la selección de variables.

Variable	Varianza	Validación	Colinealidad	Total
<i>Key y Mode</i>	0,00	Simple	–	173
<i>Loudness</i>	0,00	Simple	–	856
<i>Perceptibles (5)</i>	0,00	K-Fold ($K = 5$)	$> 0,95$	2140
<i>Speechiness</i>	0,00	Simple	$> 0,95$	92
<i>Liveness</i>	0,00	K-Fold ($K = 5$)	$> 0,95$	67
<i>Primary Mood</i>	0,01	Simple	$> 0,95$	1423
<i>Secondary Mood</i>	0,01	Simple	$> 0,95$	1397
<i>Suggested Genres</i>	0,01	Simple	$> 0,95$	1309

La columna 'Varianza', indica el umbral del filtro por varianza, 'Validación', indica el tipo de validación utilizada (Simple o K-Fold), y 'Colinealidad', indica el umbral de correlación para el filtro por dependencia lineal.

Capítulo 4

3.3. Modelado y optimización

Modelado mediante perceptrón multicapa (MLP)

Para la predicción de las cinco variables perceptibles (*acousticness*, *danceability*, *energy*, *instrumentalness*, *valence*), se implementó una red neuronal de tipo perceptrón multicapa. La arquitectura diseñada consta de tres capas ocultas con una reducción progresiva de neuronas (512, 256 y 128 neuronas, respectivamente). La capa de salida cuenta con cinco neuronas y emplea una función de activación sigmoid, para garantizar que las predicciones se acoten dentro del rango físico [0, 1].

Para la optimización de pesos se empleó el algoritmo AdamW [46]. Este método incorpora un desacoplamiento de la penalización de peso (weight decay) para favorecer la generalización. Como función de pérdida se seleccionó Huber Loss debido a su robustez frente a valores atípicos en las etiquetas subjetivas [47, 39]. Su ventaja radica en combinar el comportamiento del error absoluto medio (MAE) para penalizar los errores grandes y el del error cuadrático medio (MSE) para los errores pequeños y así asegurar una convergencia suave al acercarse a la solución óptima. Por último, el entrenamiento se ejecutó en lotes de 512 muestras.

Para el ajuste de los hiperparámetros de aprendizaje, la tasa inicial se fijó en 10^{-4} y fue gestionada dinámicamente mediante `ReduceLROnPlateau`. Este mecanismo redujo la tasa de aprendizaje a la mitad automáticamente al detectar estancamientos.

Dado el amplio espacio de variables (2 140 variables de entrada) y la alta capacidad del modelo, se implementó la siguiente estrategia de regularización para mitigar el sobreajuste [24]:

- Capas de normalización por lotes (Batch Normalization) después de cada transformación lineal ($Wx + b$) en `nn.Linear`, para evitar que los valores numéricos se disparen entre las capas y facilitar la convergencia [48].
- Inactivación aleatoria de neuronas (Dropout) con tasas de 0,30 en la primera capa oculta y 0,20 en las subsiguientes, como filtro más agresivo de ruido en las variables iniciales y proteger las representaciones más abstractas en las capas profundas [49].
- Penalización L2 (regularización de Tikhonov) integrada en el optimizador con un factor de 10^{-5} .
- Parada anticipada (Early Stopping) basada en el conjunto de validación que interrumpió el entrenamiento tras 15 iteraciones sin mejoras.

Modelado mediante Gradient Boosting (LightGBM)

Para la predicción del resto de las variables, se implementaron modelos basados en árboles de decisión potenciados por gradiente, LightGBM [22]. Esta arquitectura se seleccionó por su eficiencia computacional al procesar espacios dimensionales complejos y su capacidad para modelar relaciones no lineales sin requerir transformaciones y tratamiento de valores atípicos en las variables de entrada.

El diseño de los modelos se adaptó a la naturaleza de cada variable objetivo, con base en el preprocesamiento detallado en etapas anteriores:

- *Key / Mode, Primary, Secondary Moods y Suggested Genres*: mayor peso a las clases minoritarias mediante el parámetro `class_weight='balanced'` y se evaluó el desempeño con la métrica F1-Score macro.
- *Liveness, Speechiness*: la separación de los conjuntos de entrenamiento y validación se ejecutó de forma estratificada tras discretizar las variables en rangos probabilísticos.
- *Loudness*: la optimización evaluó el error absoluto medio (MAE), y el modelado definitivo se configuró mediante regresión estándar (L2) para minimizar el error cuadrático medio (RMSE). Esto demostró una mejora en la predicción en volúmenes extremos.

Optimización de hiperparámetros y regularización

Se implementó una búsqueda automatizada de hiperparámetros para cada objetivo mediante la librería Optuna [23]. A continuación un resumen de los parámetros y algunas decisiones relacionadas:

- Profundidad máxima de los árboles (`max_depth`) y restricción en el número máximo de hojas (`num_leaves`) para evitar el crecimiento asimétrico descontrolado. Además, se exigió un volumen mínimo de muestras por hoja (`min_child_samples` o `min_data_in_leaf`) para evitar que los modelos crearan reglas exclusivas para registros aislados [22].
- Submuestreo de filas y columnas para mejorar la generalización. En la mayoría de los modelos se utilizó el algoritmo (GBDT) combinado con un submuestreo de filas aleatorio (`subsample`). Excepto para *Loudness*, donde se implementó el algoritmo de muestreo basado en gradientes (GOSS) [22], para retener ejemplos con mayor error predictivo y descartar una fracción de las que el modelo ya aprendió.
- Penalizaciones (L1 y L2) para frenar el peso excesivo de las ramas [39]. Adicionalmente, en la mayoría de los objetivos, se exigió una mejora mínima (`min_split_gain`) antes de permitir que el árbol se dividiera.
- Early stopping, configurado entre 30 y 50 iteraciones, para interrumpir el entrenamiento cuando el modelo dejara de mejorar.

El resumen de los hiperparámetros resultantes se detalla en la tabla 3.5. Para el entrenamiento de los modelos finales, se configuró una alta cantidad de árboles (hasta 10 000 en algunos casos) y se delegó el control total al mecanismo de early stopping. Esto, combinado con tasas de aprendizaje muy bajas, mejora el desempeño [39].

TABLA 3.5. Resumen de hiperparámetros óptimos por objetivo.

Objetivo	Prof. Máx.	Hojas	Tasa de Apren.	Muestras/Hoja	L1 (α)	L2 (λ)
Key/Mode	5	140	0,0259	34	3,1690	6,9757
Loudness	10	21	0,0868	25	0,0104	0,0390
Liveness	4	15	0,0194	17	0,0107	0,3887
Speechiness	8	34	0,0102	32	0,4942	0,2947
Primary Mood	10	32	0,0208	79	0,1084	5,0239
Secondary Mood	11	35	0,0125	69	0,1443	2,6547
Genres	6	28	0,0606	21	0,9334	4,8052

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal. El sistema de inferencia fue diseñado para operar como un servicio integrable con la infraestructura existente de Plusyc. El despliegue se realizó en una instancia de cómputo sobre Oracle Cloud Infrastructure.

5.1. Conclusiones generales

3.4.1. Diseño del servicio predictivo

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo. El núcleo de procesamiento se expuso a través de una API RESTful desarrollada con FastAPI. Esta interfaz expone el ciclo de inferencia: carga del audio, extracción de variables predictivas de entrada a los modelos MLP y LightGBM, que retornan los valores de las predicciones de las características musicales.

Algunas preguntas que pueden servir para completar este capítulo:

Para garantizar la reproducibilidad y estabilidad del entorno, la aplicación fue containerizada utilizando Docker. Dada la alta exigencia computacional y de memoria del modelo preentrenado PANNS inference, se aplicaron configuraciones a nivel de contenedor para asegurar la viabilidad en producción:

- Se limitó a una instancia. Esto previene la duplicación masiva de los modelos y pesos preentrenados en la memoria RAM, para optimizar el consumo de los recursos de la máquina virtual y evitar nuevos costos [50].
- Restricción de hilos: las operaciones matemáticas requeridas para la extracción de variables se restringieron a un solo hilo (`OMP_NUM_THREADS=1`). Esto evita cuellos de botella por saturación de la CPU y bloqueos cruzados (deadlocks) durante la extracción paralela de variables [51].
- Gestión de memoria y tiempo de respuesta: se asignó un bloque de memoria compartida (`shm_size: '5gb'`) y se establecieron timeouts holgados (120 segundos) para prevenir interrupciones provocadas por el análisis de pistas de audio de larga duración, restringidas a 15 máximo máximo por canción.

5.2. Puntos por resolver

Acá se indica cómo se podría continuar el trabajo más adelante.

3.4.2. Flujo de integración

La interacción del motor predictivo con el ecosistema web se realiza a través de un flujo de procesamiento asíncrono. La comunicación entre componentes es orquestada por una herramienta de Plusyc denominada Beethoven.

El ciclo de vida de la integración, ilustrado en la figura 3.12, sigue esta secuencia lógica:

1. Una tarea programada (cron job) en el sistema ejecuta el CLI Beethoven a intervalos regulares de tiempo.
2. El proceso detecta nuevos archivos de audio que aún no han sido caracterizados.
3. Por cada nueva pista detectada, Beethoven envía el archivo al servicio de inferencia (API) mediante una petición HTTP POST.
4. **Inferencia:** La API ejecuta el pipeline predictivo, que recibe la señal cruda y retorna un diccionario estructurado (JSON) que contiene todas las etiquetas estimadas (*Key, Moods, Liveness, etc.*).
5. El script captura la respuesta de la API, extrae las características musicales y las persiste en la base de datos de información musical de Plusyc.
6. Una vez en la base de datos, los metadatos quedan disponibles para revisión de curadores en un portal web.

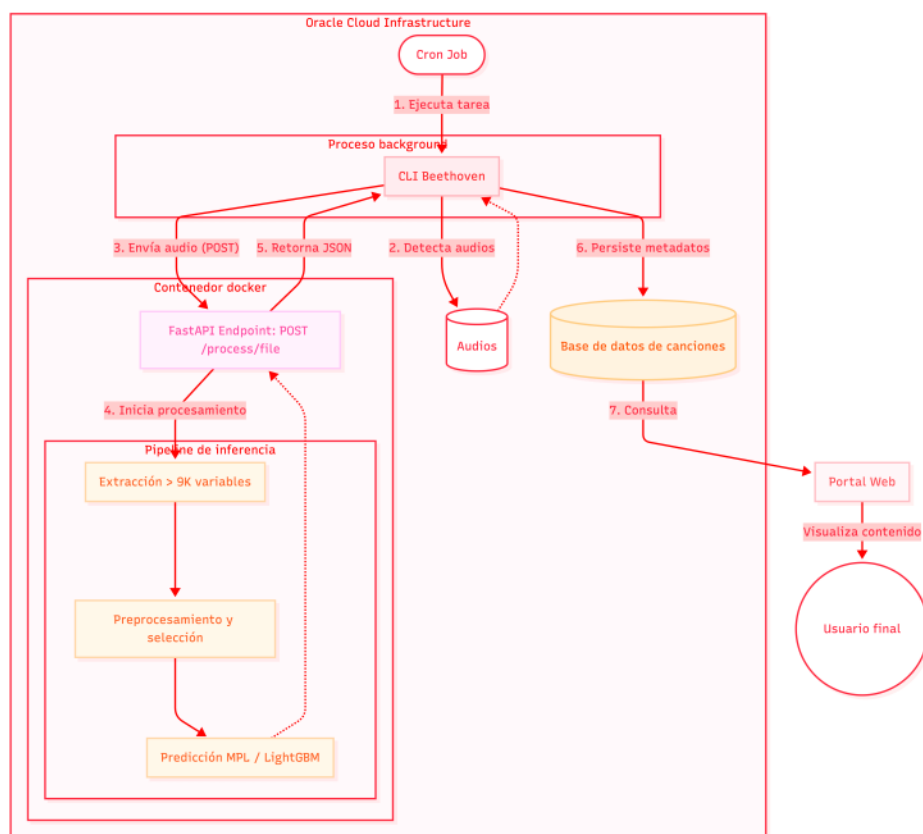


FIGURA 3.12. Diagrama de despliegue e integración.

Bibliografía

Ensayos y resultados

- [1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).
- [2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).
- [3] Gordon C. Bruner Jr. «Music, mood, and marketing». En: *Journal of marketing* (1990).
- [4] Frank A. Kardes y Reed Owsen. «Beating the tone with the tune: A meta-analysis and review of the effects of background music in retail settings». En: *Journal of Business Research* (2006).
- [5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).
- [6] Plusyc Live SAS. Perfil oficial en Instagram. 2026. URL: <https://www.instagram.com/plusyc/>.
- [7] Brian McFee et al. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. 2015. URL: <https://librosa.org/>.
- [8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*. 2013. URL: <http://essentia.upf.edu/>.
- [9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019. URL: <https://github.com/jordipons/musicnn>.
- [10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: <https://research.google.com/audioset/>.
- [11] Spotify AB. *Spotify Web API Reference: Get Audio Features*. 2026. URL: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>.
- [12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: <https://gracenote.com/products/audio-data/>.
- [13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: <https://tunebat.com/>.
- [14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).
- [15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.
- [16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

- [17] Alexander Lerch. *An introduction to audio content analysis: Applications in signal processing and music informatics*. John Wiley & Sons, 2012.
- [18] Emilia Gómez. «Tonal description of polyphonic audio for music content processing». En: *INFORMS Journal on Computing* (2006).
- [19] Christopher Harte, Mark Sandler y Martin Gasser. «Detecting harmonic change in musical audio». En: *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*. 2006.
- [20] Qiuqiang Kong et al. «PANNs: Large-scale pretrained audio neural networks for audio pattern recognition». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2020).
- [21] Karen Simonyan y Andrew Zisserman. «Very deep convolutional networks for large-scale image recognition». En: *International Conference on Learning Representations*. 2015.
- [22] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». En: *Advances in neural information processing systems*. Vol. 30. 2017.
- [23] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.
- [24] Sebastián Ramírez. *FastAPI*. 2018. URL: <https://github.com/fastapi/fastapi>.
- [25] Python Software Foundation. *Python Language Reference*. 2026. URL: <https://www.python.org/>.
- [26] Oracle Corporation. *Oracle Cloud Infrastructure*. 2026. URL: <https://www.oracle.com/cloud/>.
- [27] Thierry Bertin-Mahieux et al. «The million song dataset». En: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.
- [28] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* (1998).
- [29] Bob L. Sturm. «The state of the art ten years after a state of the art: Future research in music information retrieval». En: *Journal of New Music Research* (2014).
- [30] Geoffroy Peeters. *A large set of audio features for sound description (similarity and classification) in the CUIDADO project*. Inf. téc. IRCAM (Institut de Recherche et Coordination Acoustique/Musique), 2004.
- [31] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. 2020. URL: <https://www.kaggle.com/datasets/andradolteanu/gtzan-dataset-music-genre-classification>.
- [32] Shawn Hershey et al. «CNN architectures for large-scale audio classification». En: *IEEE*. 2017.
- [33] ITU-R. «Algorithms to measure audio programme loudness and true-peak audio level». En: *Recommendation ITU-R BS.1770-4* (2015).
- [34] Alain De Cheveigné e Hideki Kawahara. «YIN, a fundamental frequency estimator for speech and music». En: *The Journal of the Acoustical Society of America* (2002).
- [35] Earl Vickers. «The loudness war: Background, speculation and recommendations». En: *Audio Engineering Society*. 2010.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).
- [2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).
- [3] Gordon C Bruner II. «Music, mood, and marketing». En: *Journal of marketing* (1990).
- [4] Franck V Garlin y Keri Owen. «Setting the tone with the tune: A meta-analytic review of the effects of background music in retail settings». En: *Journal of Business Research* (2006).
- [5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).
- [6] Plusyc Live SAS. *Perfil oficial en Instagram*. 2026. URL: <https://www.instagram.com/plusyc/>.
- [7] Brian McFee et al. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. 2015. URL: <https://librosa.org/>.
- [8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*. 2013. URL: <http://essentia.upf.edu/>.
- [9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019. URL: <https://github.com/jordipons/musicnn>.
- [10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: <https://research.google.com/audioset/>.
- [11] Spotify AB. *Spotify Web API Reference: Get Audio Features*. 2026. URL: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>.
- [12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: <https://gracenote.com/products/audio-data/>.
- [13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: <https://tunebat.com/>.
- [14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).
- [15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.
- [16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

- [17] Alexander Lerch. *An introduction to audio content analysis: Applications in signal processing and music informatics*. John Wiley & Sons, 2012.
- [18] Emilia Gómez. «Tonal description of polyphonic audio for music content processing». En: *INFORMS Journal on Computing* (2006).
- [19] Christopher Harte, Mark Sandler y Martin Gasser. «Detecting harmonic change in musical audio». En: *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*. 2006.
- [20] Qiuqiang Kong et al. «PANNs: Large-scale pretrained audio neural networks for audio pattern recognition». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2020).
- [21] Karen Simonyan y Andrew Zisserman. «Very deep convolutional networks for large-scale image recognition». En: *International Conference on Learning Representations*. 2015.
- [22] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». En: *Advances in neural information processing systems*. Vol. 30. 2017.
- [23] Takuya Akiba et al. «Optuna: A next-generation hyperparameter optimization framework». En: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019.
- [24] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.
- [25] Sebastián Ramírez. *FastAPI*. 2018. URL: <https://github.com/fastapi/fastapi>.
- [26] Python Software Foundation. *Python Language Reference*. 2026. URL: <https://www.python.org>.
- [27] Dirk Merkel. «Docker: lightweight linux containers for consistent development and deployment». En: *Linux journal* ().
- [28] Oracle Corporation. *Oracle Cloud Infrastructure*. 2026. URL: <https://www.oracle.com/cloud/>.
- [29] Thierry Bertin-Mahieux et al. «The million song dataset». En: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.
- [30] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* (1998).
- [31] Bob L. Sturm. «The state of the art ten years after a state of the art: Future research in music information retrieval». En: *Journal of New Music Research* (2014).
- [32] Geoffroy Peeters. *A large set of audio features for sound description (similarity and classification) in the CUIDADO project*. Inf. téc. IRCAM (Institut de Recherche et Coordination Acoustique/Musique), 2004.
- [33] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. 2020. URL: <https://www.kaggle.com/datasets/andradolteanu/gtzan-dataset-music-genre-classification>.
- [34] Shawn Hershey et al. «CNN architectures for large-scale audio classification». En: *IEEE*. 2017.
- [35] ITU-R. «Algorithms to measure audio programme loudness and true-peak audio level». En: *Recommendation ITU-R BS.1770-4* (2015).
- [36] Alain De Cheveigné e Hideki Kawahara. «YIN, a fundamental frequency estimator for speech and music». En: *The Journal of the Acoustical Society of America* (2002).

- [37] Spotify AB. *Spotify Web API Reference: Get Track Audio Features*. Accedido en el transcurso de la investigación. 2024. URL: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>.
- [38] Earl Vickers. «The loudness war: Background, speculation and recommendations». En: Audio Engineering Society. 2010.
- [39] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.
- [40] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [41] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* (2011).
- [42] Max Kuhn y Kjell Johnson. *Applied Predictive Modeling*. Springer, 2013.
- [43] Ron Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». En: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*. Morgan Kaufmann Publishers Inc. 1995.
- [44] Leo Breiman et al. *Classification and Regression Trees*. CRC press, 1984.
- [45] Isabelle Guyon y André Elisseeff. «An introduction to variable and feature selection». En: *Journal of Machine Learning Research* (2003).
- [46] Ilya Loshchilov y Frank Hutter. «Decoupled Weight Decay Regularization». En: *International Conference on Learning Representations*. 2019.
- [47] Peter J Huber. «Robust estimation of a location parameter». En: *The Annals of Mathematical Statistics* ().
- [48] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International Conference on Machine Learning*. PMLR.
- [49] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The Journal of Machine Learning Research* (2014).
- [50] Mark Treveil et al. *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media, 2020.
- [51] Micha Gorelick y Ian Ozsvald. *High Performance Python: Practical Performant Programming for Humans*. O'Reilly Media, 2020.